



JMPM TECNOLOGIA

TECNOLOGIA & DESENVOLVIMENTO

GUIA RÁPIDO

Como Começar

Guia rápido para colocar o Motor Core GraphQL para Protheus em funcionamento

Versão **3.1.0**

JMPM Tecnologia

05/09/2026

Sumário

Como Começar

O que é este projeto

Pré-requisitos

Passo 1 — Compilar e implantar

Passo 2 — Testar a introspecção

Passo 3 — Ver os campos de uma tabela

Passo 4 — Fazer sua primeira consulta

Passo 5 — Fazer sua primeira mutation (opcional)

Onde ir a partir daqui

Guia Rápido — Guia rápido de início: compilar, implantar e executar a primeira consulta em poucos minutos.

Como Começar

Guia rápido para colocar o motor GraphQL do Protheus em funcionamento e executar sua primeira consulta em poucos minutos.

O que é este projeto

Um servidor GraphQL nativo em TLPP que expõe o dicionário de dados do Protheus (tabelas SX2/SX3, relacionamentos SX9) como um schema GraphQL **dinâmico** — nenhum tipo é escrito à mão. Toda tabela liberada aparece automaticamente como um tipo consultável, com seus campos reais.

Hoje o motor cobre: - **Leitura**: consultas paginadas e filtráveis sobre qualquer tabela não bloqueada, incluindo campos de relacionamento aninhados (via SX9). - **Escrita**: mutations `create / update / delete` sobre tabelas explicitamente liberadas.

Pré-requisitos

- Um AppServer Protheus já rodando, com REST habilitado (`[HTTPREST]` no `appserver.ini`) na porta configurada (`9995` nos exemplos abaixo).
- Acesso para compilar e implantar fontes `.tlpp` nesse ambiente.

Passo 1 — Compilar e implantar

Compile todos os fontes em `custom/backoffice/graphql/core/` e `custom/backoffice/graphql/entrypoints/` e implante o RPO resultante no seu AppServer, seguindo o processo de build já usado no seu ambiente.

Atenção: o arquivo de configuração `custom/backoffice/graphql/config/graphql-config.json` **não** faz parte do RPO compilado — ele precisa ser copiado manualmente para o `RootPath` do AppServer (veja `appserver.ini`, seção `[P12] RootPath=`), não para o `SourcePath`. Sem esse arquivo no lugar certo, a lista de bloqueio fica vazia silenciosamente e **todas** as tabelas ficam visíveis — confirmado em ambiente real durante o desenvolvimento. Detalhes completos no `manual-implementacao.md`.

Passo 2 — Testar a introspecção

Com o serviço no ar, confirme que o endpoint responde:

```
curl "http://localhost:9995/rest/graphql"
```

A resposta lista os nomes de todas as tabelas liberadas (respeitando a lista de bloqueio do `graphql-config.json`).

Passo 3 — Ver os campos de uma tabela

```
curl "http://localhost:9995/rest/graphql?type=SA1"
```

Retorna todos os campos reais de `SA1` (nome, tipo GraphQL, tamanho máximo, se é obrigatório), extraídos ao vivo do SX3.

Passo 4 — Fazer sua primeira consulta

```
{ SA1(limit: 5, filter: [{field: "A1_COD", op: "eq", value: "000001"}]) {
  A1_COD
  A1_NOME
  SC5 { C5_NUM }
} }
```

Via `curl` (texto do GraphQL precisa ir codificado na URL):

```
curl "http://localhost:9995/rest/graphql?
query=%7B%20SA1(limit%3A%205)%20%7B%20A1_COD%20A1_NOME%20%7D%20%7D"
```

Passo 5 — Fazer sua primeira mutation (opcional)

Mutations só funcionam em tabelas explicitamente liberadas em `allowMutations` no `graphql-config.json` (nenhuma por padrão). Supondo que `SA1` esteja liberada:

```
mutation { createSA1(input: {A1_COD: "000123", A1_LOJA: "01", A1_NOME: "Foo"}) {
  A1_COD
  A1_NOME
} }
```

Atenção: mutations exigem a palavra-chave `mutation` no início do texto — sem ela, o servidor interpreta como uma consulta de leitura comum e retorna erro de "tabela desconhecida".

Onde ir a partir daqui

- `manual-implementacao.md` — como implantar, configurar e operar este motor em um ambiente Protheus real (passo a passo completo, incluindo a pegadinha do `RootPath` e as limitações conhecidas).
- `manual-utilizacao.md` — referência completa da linguagem GraphQL suportada: sintaxe de consultas, filtros, paginação, relacionamentos e mutations, com exemplos.
- `architecture.md` — arquitetura interna do motor, para quem for estender ou depurar o código.